Trasformazioni del nostro algoritmo zk:

May 19, 2026

Frigo Shelat

$$x = (pk, a, id, tr, now), \;\; w = (MSO, pk_{dx}, pk_{dy})$$
$$e1 = SHA256(MSO[0 : 183])$$
$$a = MSO[id]$$
$$(pkdx, pkdy) = MSO[96 : 160]$$
$$tstart = MSO[48 : 56]$$
$$tend = MSO[56 : 64]$$
$$tstart < tnow < tend$$
$$true = p256.verify((r1, s1), e1, PKII)$$
$$true = p256.verify((r2, s2), H(tr||hdr), (pkdx, pkdy))$$

Post quantum transition

Issuer's signature proof zk-dillithium (STARK):

$x = (PK_{II}, C), w = (MSO, r)$

$C = SHA256(MSO, r)$

$e1 = SHA256(MSO[0 : 183])$

$true = Dillithium.verify((r1, s1), e1, PK_{II})$

Attribute disclosure, document validity and device binding proof (LIGERO):

$x = (a, id, tr, now, C), w = (MSO, pk_{dx}, pk_{dy}, r)$

$C = SHA256(MSO, r)$

$a = MSO[id]$

$(pkdx, pkdy) = MSO[96 : 160]$

$tstart = MSO[48 : 56]$

$tend = MSO[56 : 64]$

$tstart < tnow < tend$

$true = p256.verify((r2, s2), H(tr||hdr), (pkdx, pkdy))$

Verifier additional check: STARK(C) = LIGERO(C)

Senza mostrare all'issuer la device public key

Issuer's signature proof zk-dillithium (STARK):

$x = (PK_{II}, C), w = (MSO, r)$

$C = \text{SHA256}(MSO, r)$

$e1 = \text{SHA256}(MSO[0:183])$

$true = Dillithium.verify((r1, s1), e1, PK_{II})$

Attribute disclosure, document validity and device binding proof (LIGERO):

$x = (a, id, tr, now, C), w = (MSO, pk_{dx}, pk_{dy}, r)$

$C = \text{SHA256}(MSO, r)$

$a = MSO[id]$

$SHA256(pkdx, pkdy) = MSO[96:160]$

$tstart = MSO[48:56]$

$tend = MSO[56:64]$

$tstart < tnow < tend$

$true = p256.verify((r2, s2), H(tr||hdr), (pkdx, pkdy))$

Ottimizzazione con poseidon hash

Rimpiazziamo SHA256 con Poseidon (solo su STARK Ligero già ottimizza SHA). Issuer's signature proof zk-dillithium (STARK):

$x = (PK_{II}, C), w = (MSO, r)$

$C = Poseidon(MSO, r)$

$e1 = Poseidon(MSO[0 : 183])$

$true = Dillithium.verify((r1, s1), e1, PK_{II})$

(LIGERO):

$x = (a, id, tr, now, C), w = (MSO, pk_{dx}, pk_{dy}, r)$

$C = Poseidon(MSO, r)$

$a = MSO[id]$

$SHA256(pkdx, pkdy) = MSO[96 : 160]$

$tstart = MSO[48 : 56]$

$tend = MSO[56 : 64]$

$tstart < tnow < tend$

$true = p256.verify((r2, s2), H(tr||hdr), (pkdx, pkdy))$

Nota che è possibile ottimizzare ulteriormente parallelizzando la creazione delle due prove.

Ottimizzazione del commitment

Il nuovo commitment è $C = H(MSO) \oplus r_1$. Eliminiamo di fatto una prova su SHA in zkSTARK.

Issuer's signature proof zk-dillithium (STARK):

$x = (PK_{II}, C), w = (MSO, r)$

$e1 = \text{Poseidon}(MSO[0 : 183])$

$C = e_1 \oplus r_1$

$true = Dillithium.verify((r1, s1), e1, PK_{II})$

Attribute disclosure, document validity and device binding proof (LIGERO):

$x = (a, id, tr, now, C), w = (MSO, pk_{dx}, pk_{dy}, r)$

$C = Poseidon(MSO) \oplus r_1$

$a = MSO[id]$

$SHA256(pkdx, pkdy) = MSO[96 : 160]$

$tstart = MSO[48 : 56]$

$tend = MSO[56 : 64]$

$tstart < tnow < tend$

$true = p256.verify((r2, s2), H(tr||hdr), (pkdx, pkdy))$